Project overview

EventPilot

Team and project: EventPilot
Enock Okorno Ayiku and Haley Wodarczyk
AI Applications Hackathon | Prepared October 24, 2026
Repository: https://github.com/physician1/EventPilot

Problem and intended users

Planning a small celebration requires translating an informal idea into venues, suppliers, a schedule and a spending plan. Information is scattered, quotes are incomplete, and comparing options is time-consuming. EventPilot targets individuals planning birthdays or weddings and small teams arranging corporate gatherings. These are target-user hypotheses, not findings from completed customer interviews.

Proposed solution

A local web app saves event requirements, researches the entered location, shows linked sources, and presents a provisional supplier shortlist, budget allocation and timeline. Users can revise their preferences through a conversation and review the resulting research. A separate offline rehearsal demonstrates a fictional Boston catalog.

Why agentic AI

The live researcher uses the saved brief to direct web searches and synthesize a cited plan. Search is a tool-mediated action rather than a fixed catalog lookup. This is a bounded single-agent research workflow; deterministic code handles persistence, run controls and dashboard calculations. The older catalog path has planner and critic roles, but live research does not currently run an independent critic.

Prototype status

The prototype builds and passes automated checks. It supports research and planning, not booking. Published venue facts still require confirmation. The report distinguishes software checks from unverified supplier claims and from proposed future capabilities.

Agent design and workflow

Live workflow

1 Save requirements and provide consent

2 Research agent calls web search (up to 6 tool calls)

3 Check completion and citations; show sourced report

4 Project provisional suppliers, budget and timeline

5 Human reviews, verifies and requests revisions

Responsibilities and context

The user chooses event type, city/country, date/time, duration, guest count, indoor requirement and USD budget. The researcher receives these requirements, preferences, feedback and previous research. Event name and historical approvals are excluded from the structured live request. Free-text fields may still contain personal data entered by users.

Tools and boundaries

The backend calls the OpenAI Responses API with web search required, up to six tool calls per run, a 90-second client timeout and no client retries. The model is configurable; the default is gpt-4.1-mini. Completed search activity and at least one valid HTTP(S) citation are checked before accepting the report. These checks do not verify the truth of every generated claim.

Human decisions

Users consent before live research, review sources, edit saved requirements, choose among research candidates and adjust provisional allocations. Nothing books, pays or contacts suppliers. Stop and pause controls prevent stale responses from replacing newer state, but cannot revoke a request already sent. Live research is not an approved, priced supplier package.

Separate rehearsal path

Scripted rehearsal uses fictional Boston suppliers without AI calls. The legacy model-driven catalog path uses planner/critic roles and coded comparisons. Do not present a rehearsal result as live research or claim that the live web path receives the catalog critic audit.

Implementation and working prototype

Technology

Frontend: React 19 and Vite 6. Backend: FastAPI, Pydantic, OpenAI Python SDK and SQLite. The app runs locally at 127.0.0.1:8000. Source modules and Windows setup commands are documented in the packaged README.

From research to dashboard

The frontend extracts candidate names/categories and suggested times from the report using heuristics. Overview displays provisional selections and supplier links; Discover exposes research candidates; Timeline shows parsed research times or a labeled fallback template. Missing service categories are flagged instead of presented as confirmed suppliers. Extraction can still miss or misclassify content.

Budget policy, not market pricing

The initial allocation distributes 70% of the saved budget among service categories. Fees add 15% of that subtotal; contingency adds 10% of the saved budget. For a USD 5,000 birthday, the initial example is USD 3,500 in service allocations + USD 525 fees + USD 500 contingency = USD 4,525, leaving USD 475 unallocated. This arithmetic does not demonstrate that suppliers can actually deliver within budget.

State and data

SQLite stores events and research. Provisional dashboard edits are stored in the browser, scoped to the event and research timestamp; they do not synchronize across browsers. New research creates a new projection. Export includes provisional plan and timeline data. Event deletion requires confirmation and is permanent.

Geographic and product scope

Live research has no city allowlist and accepts locations such as Accra, Ghana. Coverage is not guaranteed in every country. Scripted rehearsal remains Boston-only. Intake categories are birthday, wedding and corporate. Currency is USD; published local prices are not automatically converted.

Evaluation and evidence

Observed checks on October 24, 2026

Backend: 55 tests passed in 11.39 seconds, with one Starlette/httpx deprecation warning. Frontend projection: six tests passed. Vite production build passed in 26.31 seconds. These 61 automated checks are not 61 live model trials. Provider tests use mocked responses; no fresh paid live-model evaluation was performed for this submission package.

Representative expected and actual outcomes

1. Missing research service: expected an explicit missing-supplier indication; frontend test passed.
2. Edited allocations: expected recalculation and over-budget detection; frontend test passed.
3. Unsafe supplier URL: expected removal; frontend test passed.
4. Recognized research schedule: expected extracted timeline; frontend test passed.
5. No usable schedule: expected a labeled fallback template; frontend test passed.
6. Invalid saved numeric edits: expected finite, usable totals; frontend test passed.

Manual example from development

The user supplied an Accra birthday run with 40 guests, indoor requirements and a USD 5,000 budget. It returned a cited shortlist and suggested schedule. This demonstrates an observed location-specific research response, not verified venue availability, supplier suitability or quote accuracy. Earlier Boston-only catalog runs returned zero Accra options; separating live search from rehearsal addressed that mode mismatch.

Failures that shaped the design

Earlier runs showed inconsistent budget narration, generic validation/provider failures, raw report formatting, misleading decision counters and a blank Overview after live research. Current code preserves structured saved requirements, renders research, distinguishes research completion and projects a provisional dashboard. Provider outages and malformed responses can still block a run. Heuristic extraction and quote verification remain open issues.

Next evaluation

Run and record fresh live trials across several locations and event types. Human reviewers should score citation support, supplier relevance, coverage of required services, indoor/capacity evidence, response time and cost. Record failures as well as successes. Compare against manual search using the same briefs. No such comparative performance claims are made here.

Positioning and responsible AI

Target market and alternatives

Initial focus: people organizing small private celebrations and small teams planning gatherings. Direct overlap exists with Cvent Vendor Marketplace, which advertises vendor sourcing, AI-assisted matching and quote comparison [1]. Peerspace overlaps in venue discovery and booking [2]. Indirect alternatives include manual web search, spreadsheets and professional planners. This is desk research, not a market-size estimate or validated willingness-to-pay study.

Value proposition

EventPilot combines a saved brief, conversational research, visible sources and an editable provisional planning view in a local prototype. Its proposed value is reducing the work of assembling an initial plan while keeping uncertainty visible. AI sourcing itself is not unique; established competitors already offer related features. The next product test is whether this particular workflow helps small-event users make better shortlists.

Responsible use

Require consent for live data sharing, bound calls, expose citations, label fictional rehearsal suppliers and provisional allocations, and reserve decisions for the user. Render a safe subset of report formatting and restrict source links to HTTP(S). Do not describe citations as fact-checking or a complete defense against malicious web content.

Remaining risks

Incorrect or outdated supplier facts; weak coverage of some locations; irrelevant large venues; missing service categories; unverified costs and capacity; network/provider failures; local-storage loss; and personal data in free text. The local app lacks production authentication and multi-user isolation. Published prices need currency-aware handling and written supplier confirmation.

Future vision

Prioritize structured research outputs and evidence per supplier attribute, followed by quote-aware budgeting, broader location evaluation and a separate evidence-checking critic. Add durable synchronized edits and production access controls before wider deployment. Booking, supplier outreach and payment integrations are future proposals, not implemented capabilities.

Demo, reproduction and references

Demonstration story

Show a person planning an indoor birthday in Accra for 40 guests with USD 5,000. Save the requirements, submit a research request after consent, and explain the visible tool-backed response. Open a supplier source, inspect the provisional Overview and Timeline, then revise a preference. Explain that the allocation is not a quote and nothing is booked. Use a clearly labeled Boston rehearsal if the provider is unavailable.

Reproduce the prototype

The source package contains the latest frontend build, backend, tests, dependency files and Windows README. Create a Python virtual environment, install requirements.lock, copy .env.example to .env and optionally add your own API key. Run uvicorn backend.main:app on 127.0.0.1:8000. For a GitHub clone, build the frontend with npm ci and npm run build -- --configLoader native. Run pytest and npm test for checks.

Repository and submission access

Repository: https://github.com/physician1/EventPilot. The repository is public and accessible to reviewers without a collaborator invitation. No credentials, personal event database or virtual environment is included. Team photos, manually designed slides and horizontal poster remain team-supplied deliverables. The assignment accepts PDFs or view-access links.

References

[1] Cvent, Vendor Marketplace:
https://www.cvent.com/en/event-marketing-management/vendor-marketplace
[2] Peerspace, How do I book a space?:
https://support.peerspace.com/en/articles/10119133-how-do-i-book-a-space
[3] OpenAI, Web search guide: https://developers.openai.com/api/docs/guides/tools-web-search
[4] AI Applications Hackathon Project assignment supplied by the team.
[5] EventPilot source and automated checks, local snapshot, October 24, 2026.
Competitor pages reviewed October 24, 2026. No supplier endorsements or verified quotes are asserted.

Acknowledgment

AI assisted code development, troubleshooting and preparation of this written submission. The team must review the claims and take responsibility for the final work. Presentation and poster guidance is writing assistance only; the assignment prohibits AI-generated slides. Enock Okorno Ayiku and Haley Wodarczyk should manually design those deliverables and use authentic images.